

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr. Hemavathi R

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5

9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I Sree (192524023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sree B

1. Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| student_id | name  | age  | Department_id |
+-----+-----+-----+-----+
|          101 | Rahul | 20   |          101 |
|          102 | Priya | 21   |          102 |
|          103 | Arun  | 22   |          103 |
|          104 | Kiran | 20   |          104 |
|          105 | Neha  | 21   |          105 |
+-----+-----+-----+-----+
5 rows in set (0.010 sec)
```

2. The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

```
mysql> CREATE TABLE Employee (
->     emp_id INT PRIMARY KEY,
->     emp_name VARCHAR(30),
->     salary INT,
->     Department_id INT,
->     FOREIGN KEY (Department_id) REFERENCES Department(Department_id)
-> );
Query OK, 0 rows affected (1.288 sec)

mysql> INSERT INTO Employee VALUES
-> (1, 'Rahul', 40000, 101),
-> (2, 'Priya', 50000, 101),
-> (3, 'Arun', 35000, 102),
-> (4, 'Neha', 60000, 102),
-> (5, 'Kiran', 45000, 103);
Query OK, 5 rows affected (0.207 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| emp_id | emp_name | salary | Department_id |
+-----+-----+-----+-----+
|        1 | Rahul    | 40000  |          101 |
|        2 | Priya    | 50000  |          101 |
|        3 | Arun     | 35000  |          102 |
|        4 | Neha     | 60000  |          102 |
|        5 | Kiran    | 45000  |          103 |
+-----+-----+-----+-----+
5 rows in set (0.034 sec)
```

3. A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

```
mysql> SELECT * FROM Department_View;
+-----+-----+-----+
| Department_id | Departmem_name | block |
+-----+-----+-----+
|          101 | CSE            | AHS   |
|          102 | ECE            | ECE Block |
|          103 | Civil          | SCAD  |
|          104 | IT             | ADMIN |
|          105 | Bio-Tech       | RHS   |
+-----+-----+-----+
5 rows in set (0.044 sec)
```

```
mysql> CREATE OR REPLACE VIEW Department_View AS
-> SELECT
->     Department_id,
->     Departmem_name,
->     block
-> FROM Department;
Query OK, 0 rows affected (0.102 sec)
```

```
mysql> SELECT * FROM Department_View;
+-----+-----+-----+
| Department_id | Departmem_name | block |
+-----+-----+-----+
|          101 | CSE            | AHS   |
|          102 | ECE            | ECE Block |
|          103 | Civil          | SCAD  |
|          104 | IT             | ADMIN |
|          105 | Bio-Tech       | RHS   |
+-----+-----+-----+
5 rows in set (0.015 sec)
```

4. Debug the given relational algebra expression that produces incorrect output for a natural join operation.

```
mysql> SELECT e.student_id,
->     c.course_id,
->     c.course_name
-> FROM Enroll e
-> JOIN Course c
-> ON e.course_id = c.course_id;
```

student_id	course_id	course_name
101	1	DBMS
101	2	Java
102	2	Java
103	3	Python

```
4 rows in set (0.377 sec)
```

5. The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints

```
mysql> SELECT * FROM Employee;
```

emp_id	emp_name	salary	Department_id
1	Rahul	40000	101
2	Priya	50000	101
3	Arun	35000	102
4	Neha	60000	102
5	Kiran	45000	103
6	Rahul	40000	110

```
6 rows in set (0.009 sec)
```



```
mysql> SELECT * FROM Department;
```

Department_id	Department_name	numberofcourses	credits	block
101	CSE	32	192	AHS
102	ECE	32	192	ECE Block
103	Civil	30	190	SCAD
104	IT	32	192	ADMIN
105	Bio-Tech	32	192	RHS
110	Mechanical	30	180	ME Block

```
6 rows in set (0.010 sec)
```



```
mysql>
```

6. Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance

```
mysql> SELECT DISTINCT
->      c.customer_id,
->      c.customer_name,
->      c.city
-> FROM Customer c
-> JOIN Orders o
-> ON c.customer_id = o.customer_id;
```

customer_id	customer_name	city
1	Rahul	Chennai
2	Priya	Bangalore
4	Neha	Delhi

```
3 rows in set (0.398 sec)
```



```
mysql> |
```

7. The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

```
mysql> SELECT product_name,
->      SUM(quantity) AS Total_Quantity
-> FROM Sales
-> GROUP BY product_name;
```

product_name	Total_Quantity
Pen	30
Book	25
Pencil	25

3 rows in set (0.011 sec)

8. Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table

```
mysql> SELECT * FROM Student;
```

student_id	name	age	Department_id
101	Rahul	23	101
102	Priya	21	102
103	Arun	22	103
104	Kiran	20	104
105	Neha	21	105

5 rows in set (0.009 sec)

9. Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

```
mysql> SHOW INDEX FROM Orders;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index
orders	0	PRIMARY	1	order_id	A	4	NULL	NULL		BTREE		
orders	1	idx_orders_customer	1	customer_id	A	3	NULL	NULL	YES	BTREE		

2 rows in set (0.546 sec)

```
mysql> SHOW INDEX FROM Payment;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
payment	0	PRIMARY	1	payment_id	A	3	NULL	NULL		BTREE		
payment	1	idx_payment_order	1	order_id	A	3	NULL	NULL	YES	BTREE		

```
2 rows in set (0.182 sec)
```

10. Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

```
mysql> SELECT * FROM Student;
```

student_id	name	age	Department_id
101	Rahul	23	101
102	Priya	21	102
103	Arun	22	103
104	Kiran	20	104
105	Neha	21	105

```
5 rows in set (0.008 sec)
```

```
mysql> SELECT s.student_id,
->      s.name,
->      d.Department_name
-> FROM Student s
-> JOIN Department d
-> ON s.Department_id = d.Department_id;
```

student_id	name	Department_name
101	Rahul	CSE
102	Priya	ECE
103	Arun	Civil
104	Kiran	IT
105	Neha	Bio-Tech

```
5 rows in set (0.011 sec)
```

```
mysql> |
```